



摘要：

“下一个时代并不是 Claude Code时代。”

“它会逐渐退到软件背景中，像数据库、正则表达式或其他基础设施一样，成为开发者随手调用的普通能力。”

这是 TypeSafe CEO、Jev 创始人 Diogo Almeida 对 AI 的设想：当智能真正融入软件，用户甚至不必意识到它的存在。但怎样才能让开发者放心地把判断交给 AI，而不用守在旁边反复检查？

几个月前，在 AI Engineer 大会的演讲中，这位曾参与 OpenAI InstructGPT 工作的研究者就提出，“下一个时代并不是 Claude Code时代”。在他看来，Claude Code 与 Codex 本质上仍属于同一个阶段——AI 的主要角色仍然是围绕人提供辅助。他关心的是，为什么 AI 如此聪明，能解决数学界的千禧年难题，却连最基础的活都无法实现自动化工作？

9月22日，做客 Latent Space 播客接受采访时，Diogo 再次谈起这份不满，措辞更加直接：“在我看来，Jev 出现之前的 AI 世界，简直是个悲剧。”在他看来，问题在于：强大的“智能引擎”已经有了，把它接进实际业务流程的接口却仍然欠缺。

这一次，他带来了自己的答案——Jev。通过面向校准决策的强化学习（RLCD），团队希望让模型输出可供代码直接使用的选择、评分和概率，让开发者能够依据不确定性设置阈值，决定何时执行、何时交给给人处理。

从参与训练听懂人类指令的模型，到尝试让代码直接调用智能，Diogo 为什么要重新选择训练目标？他又准备如何让 AI 成为像数据库一样普通的软件能力？这场两小时的访谈讲述了他的判断与尝试。

太长不看版：

Swyx：对于那些可能不太了解情况或者只想得到确切答案的人来说，Jev 到底是什么？

Diogo Almeida：目前最准确的称呼是 System1 模型，它的能力范围远远超出决策本身。这类模型的目标，在于让代码成为模型输出的直接使用者。

Swyx：你对 RLHF 的一个核心看法是：模型的回答会向用户想听的内容，而不是反映它对一件事真正的内部置信度。能具体谈谈吗？

Diogo Almeida：几乎没人注意到 RLHF 的缺点，特别是“模式坍缩”(mode dropping)。RLHF 的模式坍缩会让模型偏向生成更安全、更常见的答案，而牺牲概率分布的真实校准，这既掩盖了长文本中的错误累积，也解释了为什么文本模型不擅长决策。

Swyx：RLCD 与 RLHF 的核心区别是什么？

Diogo Almeida：区别首先在于优化目标：RLHF 侧重让模型遵循人的指令、给出获得认可的回答，RLCD 则希望模型成为软件能够可靠调用的能力。

Swyx：为什么 Jev 不把拒答机制直接放进模型底层？



Diogo Almeida：我不反对安全本身，但我认为不应把特定价值判断直接写进通用模型底层能力，而应像数据库一样划清技术能力与具体使用责任的边界。

Swyx：你所说的可靠性是什么？开发者能否相信同一个版本的行为保持稳定？

Diogo Almeida：我更重视稳健性：问题含义没变，就不该因为加入无关字符而大幅改变判断，而不只是追求相同输入得到相同输出。已经部署的模型，我们不会悄悄修改，因为 API 是别人程序里的依赖。但我们会快速发布新版本，目前还不能承诺永久维护每个旧版本。

Swyx：不依赖公开榜单，你们怎样判断模型是否真正做到了更高的性价比？

Diogo Almeida：我们会通过内部评测比较成本和能力，追求同等成本下更强、同等能力下更便宜。我不反对评测，反对的是围绕榜单优化，让分数脱离真实价值。开发者最终还是要将模型放进自己的工作流，测量实际表现，不能只看价格、速度或一个总分。

Swyx：开发者应该怎样组织任务，才能更可靠地使用 Jev？

Diogo Almeida：我建议把复杂任务拆成最小、可以独立判断的语义单元，用结构化输入提供必要信息，再由代码控制最终行为。Choice 对应选择分支，Noul 对应条件判断，Score 对应评分、排序和筛选。每一步都可以单独评估、调整阈值；能力不足时，就转交人工或暂不部署。

Swyx：对企业开发者来说，Jev 有哪些值得尝试的应用方向？

Diogo Almeida：我们梳理了四类：分析因处理成本太高而闲置的“暗数据”；为实时流程提供快速判断；检查其他模型的调用和输出；把智能判断嵌入软件核心逻辑。我尤其看好暗数据分析和编程 Agent，但具体如何组合模型，仍要看它能否降低成本、解决原本解决不了的问题。

Swyx：面对前沿 AI 的风险，放慢发展速度是唯一选择吗？

Diogo Almeida：我认为，这种讨论往往默认大家必须沿着现有 RLVR 路线继续加大投入，但研究目标和技术路线都可以重新选择。为了提升表现而给模型多大行动空间，本身就是设计决定，不该被当作不可避免的前提。我更希望探索其他方向，让已有智能可靠地进入软件，自动化实际工作。

Swyx：如果研究者在前沿实验室得不到资源，你会建议他们出来创业吗？

Diogo Almeida：要看为什么离开。如果只是想自由尝试课题，现有实验室可能仍然最合适；如果找到了值得长期投入的新任务，我会支持创业。我不认为漂亮的研究履历会自动创造价值，也不看好拿到资金后重复已有工作的做法。先明确自己的核心目标，再围绕真正值得解决的问题展开研究。

1Jev 发布的第一周，创始人的情绪糟糕透了

Swyx：欢迎来到录音室。就在本周，我的好朋友 Diogo 发布了 Jev，它几乎占领了社交平台的热点话题。此时此刻，你感觉怎么样？

Diogo Almeida：情绪上来说，从来没有这么糟过。我现在简直像一具疲惫不堪的行尸走肉，因为同时发生的事情太多了，到处都有问题等着我处理。



但是，从心理层面来说，那反而完全不一样。我经常讲这件事，并且这几年我在那些大小项目活动里也反复说过，整个 AI 领域就像游乐园里的哈哈镜屋，所有人都像疯了一样。每个人都在说各种奇怪又说不通的话。

可是就这一周，我好像和现实更合拍了，像是突然之间感受到：“哦，大家终于看见了。”——AI 能做到的事情，远比过去人们想象的多。我们真的可能推动了一场由 AI 驱动的经济革命，这件事重新回到桌面上了，简直实在是太棒了。我特别兴奋的一点，是开发者真的能够理解我们在做什么，这种感觉非常强烈。我也想对这些开发者表达我长久的感激。我对整个开发者社区以及现在发生的一切都特别兴奋，真的太棒了。

Swyx：你昨天还跟我说，你决定优先去做 town hall，也就是社区公开交流，而不是把时间都花在 VIP 和投资人之类的人身上。因为你希望确保真正得到你最多注意力的，是工程师、开发者这些真正使用产品的人。

Diogo Almeida：是的。当时确实会有一种感觉，像是“天啊，我现在正在跟一些非常重要的人说话。”我可能不该透露是谁。但对我来说，如果在我那个列满了待聊对象的庞大日程表里，开发者社区竟然不在其中，这对我来说会很不舒服。实际上，如果按照我的理想状态，我会一直和开发者社区在一起交流。我刚才甚至在想，“我要不要一边走来你的演播室，一边开一场社区大会？”后来我又觉得，“不行，这也太疯狂了。”

Swyx：对于那些可能不太了解情况或者只想得到确切答案的人来说，Jev 到底是什么？

Diogo Almeida：这个问题其实挺难回答的，但我是这么看这件事的：我们需要一种全新的模型类别。至于这一类别到底叫什么，我们没有执着于某个名字。

目前我们想到最准确的称呼是 System 1 模型。之所以没有把它称为“决策模型”，是因为 System 1 的能力范围远远超出决策本身。我现在只能说这么多。我们原本没想到这次会成为一次这么受关注的发布，所以手里还有东西没有拿出来。

Swyx：你们当时就该说这是一次“低调的研究预览”。

Diogo Almeida：某种程度上确实就是，它其实真的有点像一个研究预览。总之，我们内部有几个词来描述出现的这一类新的模型。比如机器原生模型（machine-native）、System 1 模型、大型可编程模型（large programmable）。**我的理解是，这类模型的目标，在于让代码成为模型输出的直接使用者。**

预训练大语言模型最初面向的是互联网文本补全；经过 RLHF 训练的聊天和指令遵循模型，面向的是文本回复；RLVR 则和 RLHF 之间有一块界限不太清晰的区域。而我们希望这类模型的输出能够直接被代码消费，所以公司才会叫 TypeSafe。

我们真正想要的，是让 AI 尽可能强大。而我们认为，实现这一点的方式是让它与软件结合。因此，我们设计时考虑的不只是模型外部的使用方式，连模型深层的内部机制也要针对软件进行优化。Jev 是我们的第一个大型可编程模型，也可以叫 System 1 模型，随你称它为什么。它的优化目标是“每美元智能”，Jev 这个名字也由此而来。

Swyx：Jevons Paradox，杰文斯悖论。

Diogo Almeida：对，就是杰文斯悖论。它的目标是实现性价比最高的智能模型。我特别喜欢跟人讨论，在可靠性、成本、校准和速度之间，到底什么最重要。Jev 这个名字以后会代表一系列



站在“每美元智能”方面处于领先地位的前沿模型。当然还有别的优化方向。在机器学习里，至少对于擅长机器学习的人来说，一切都关乎取舍。而我们决定在这个方向上全力以赴。

2从模式坍塌到校准失真：RLHF 的另一面

Swyx：我觉得“校准”是近来才开始受到关注的一个问题。我们之前请 Hugging Face 的 Clementine Foreia 做过一期节目，当时有聊到了这个话题。这也是你对 RLHF 的一个核心看法：模型的回答会向用户想听的内容，或者最可能出现的内容收拢，而不是反映它对一件事真正的内部置信度。

Diogo Almeida：我听说你们的听众技术背景很强，所以正想深入讲讲这个问题。发布视频里的每一项表述，我都花了很大力气核对，确保准确、真实。显然，这种做法还挺少见。视频里有一点几乎没人注意到，就是 RLHF 的缺点，尤其是“模式坍塌”(mode dropping)。

Swyx：mode dropping 还是 mode collapse？

Diogo Almeida：在这里我说的是一回事。以后我想专门写一篇博客，但现在我想尽可能把这件事讲给更多人听。我其实很认同 Yann LeCun 的不少判断。但他有一页很有名、也很有争议的幻灯片，大意是“大语言模型注定行不通”。

Swyx：你说的是那个“蛋糕”比喻？

Diogo Almeida：不是，是讲序列长度的那一页。他的推理是：如果生成每一步都有出错概率，文本越长，至少犯一次错的概率就越高。这个推理看起来在数学上很直观，但模型的实际表现并没有简单地照这个趋势发展。我很喜欢拿它来问：数学推导和观察结果之间，差异出在哪里？

Swyx：问题出在哪里？

Diogo Almeida：问题在于，如果模型试图覆盖整个分布，或者它的概率分布经过了良好校准，那么它不会因为产生少数离群结果而受到过度惩罚。你会预期它有时生成落在常见分布内的内容，有时生成分布之外的内容；覆盖整个分布，就会出现这种情况。你可以想想生成对抗网络出现之前的图像生成模型：它们生成的图像往往是模糊的。

而生成对抗网络会模式坍塌：它会直接丢掉占比很小的类别，只生成那些最常见的类别。也正因为如此，前面那个“序列越长错误必然累积到不可用”的效应并没有按最简单的方式出现。为了生成很长、又不容易出现明显错误的文本，模型就得极其保守，因为一旦犯错，人很容易看出来；相反，一个看上去没问题、其实遗漏了微妙之处的回答，就很难被察觉。为了让模型的概率分布保持校准而带来的要求，对文本序列的生成方式会产生很大的影响。这层关系很微妙。我认为，它既解释了为什么那种直观的错误累积推论没有应验，也解释了为什么文本模型不擅长决策：让本来用于生成文本的模型承担过多决策任务，效果往往不好。

Swyx：既然说到 Yann，你认同他的解决方案吗？也就是用世界模型，比如 JEPA 这一类嵌入模型的方法。问题的一部分似乎在于，我们让模型基于已经输出的词元（token）继续推理，再把输出送回去，反复循环，直到生成完整的一句话。Yann 提出的解法是联合嵌入预测架构（JEPA），你觉得这就是解决办法吗？你对此有什么看法？

Diogo Almeida：我可能不该把机器学习内部的事讲得太细。不过，除了说话有时比较放得开，我做事其实很务实。刚才对模型的判断也是从实际效果出发。



我是不是 Scaling Law 的支持者？要看它能带来什么。Scaling Law 告诉你：投入一定量的资源，某项能力能提高多少。通常，资源投入要大幅增加，收益却不会同比增长。除非那一点能力提升非常 valuable，否则看起来就不是一笔好投资。对我来说，更重要的问题是：用手头已有的资源，我们怎样才能带来最大的实际改变？

我的出发点始终是实用性。比如 Yann LeCun 的 JEPA 方向，我觉得早期研究很精彩，也很喜欢看到优秀的研究工作。至于它现在是否已经足够实用，我暂时不想下判断。

目前研究界还有很多没有被充分挖掘的好成果，像未经打磨的钻石。它们没有进一步变成有用的技术，部分原因是大家还没找到适合发挥其价值的任务。

Jev 的发布当然对 TypeSafe 有利，但我希望它带来的影响不止于此。一方面，开发者可能会基于 Jev 做出大量新软件；另一方面，也会有人沿着不同方向探索：还能用什么方式把模型能力提供给程序，让软件做出更多以前做不到的事。如果这些尝试同时出现，那种感觉就有点像早期互联网那种充满活力的氛围——大家纷纷探索，新的用法不断冒出来。我想这也是为什么推特上大家一提到“Jev”，就像在开派对一样。

Swyx：这很让人振奋，因为它和我们习惯听到的说法太不一样了。过去常有人说：“抱歉，这件事你做不了。模型研发要遵循缩放规律，只有大实验室才有资源参与。”

3Jev 的开发理念：AI 做决定时，不能突然拒答

Diogo Almeida：Discord 上经常有人问我：为什么反对安全对齐，为什么 Jev 不会拒答？我还没来得及及完整解释。先说清楚，我并不反对安全本身。我认为，常见的安全对齐方式往往与使用者的目标不一致；而对供程序调用的模型来说，突然返回一句拒绝，简直像是发生了类型错误。

如果你是人在和聊天机器人互动，或者用编程工具时碰到“抱歉，我不能读取 DNA.py”，当然会觉得烦，但至少你能看见它、换个办法继续。大家用久了，甚至习惯了处理这种情况。可如果模型是后台运行的软件依赖，某次调用突然拒绝了，会发生什么？调用它的用户可能根本不知道底下还有这个模型。难道只因为输入里出现一条特殊消息，就让整个软件流程随机中断吗？

我认为，这种设计沿用了“AI 是一个聊天同事”的想象，没有充分考虑模型作为软件组件时需要满足什么要求。

Swyx：你想要的是一种能在各种地方使用的基础能力。

Diogo Almeida：对，一个足够通用的“认知核心”。它要能适应未来各种我们现在想不到的用例。用户已经拿 Jev 做了不少出乎意料的东西，我们当然没针对那些具体应用训练过它。不过，这并不让我意外；我们训练时就让它处理过更多样的情况。

再回到安全对齐。我认为，对 ChatGPT、Claude 这种直接面向人的产品，设置安全规则是可以理解的。但需要区分能力对齐和安全对齐：能力对齐是让系统尽可能按使用者的要求完成任务。软件工程师很需要这一点——行为越可预测，越容易把模型集成进程序，也越少需要反复试探。

Jev 现在离理想状态还很远。我们希望把可靠性再提高几个数量级，最终让调用智能像执行数据库查询一样自然：需要时调用，不必每次都担心它会怎样回应。



而我说出的安全对齐，往往意味着模型除了遵循当前使用者的指令，还要优先服从模型提供方——比如 OpenAI 或 Anthropic——设定的另一套规则。这两套要求有时会发生冲突。

Swyx：你说的是不同层级的规则和价值取舍。

Diogo Almeida：对。对于直接面向用户的产品，这样做有道理。比如 ChatGPT 的提供方不希望产品支持某些成人角色扮演内容，那是他们的选择，也可能符合他们面向家庭用户的产品定位。

但 API 不一样。开发者需要根据接口的行为来写程序。如果模型在程序运行过程中突然按提供方的另一套规则拒绝执行，开发者很难处理。我对此确实有点激动，得让自己冷静一下。

Swyx：大家能感受到你很在意这件事。不过我想提出一个更严肃的反例：如果有人用这个 API 来杀人呢？成人内容或许属于私人选择，但 AI 也可能被用于战争。公司完全可以合理地决定，不希望自己的 API 被用于这类用途。

Diogo Almeida：我理解。企业可以出于实际考虑持有这种立场。我自己也不希望我们的技术被用来伤害人，当然希望它更多地用于有益的事情，也愿意为此作出努力。但我不认为应该把这些偏好直接写进通用技术的底层能力。

我的顾虑是，每当你为了某一种特定规则，强行让模型在某些问题上改变判断，就可能进一步割裂它的能力。现在的模型已经有不少这样的不一致了。

我更倾向于把智能看作数据库，而不是一位“AI 同事”。我们通常不会要求数据库在执行查询时，自行判断用户最终会不会把结果用于坏事。我认为，对通用智能接口也应该考虑类似的责任边界。

还有一件事让我觉得奇怪：有开发者在 Slack 上告诉我们准备部署某个应用，问“我们可以这样用 Jev 吗？”我的第一反应是：我们提供的是 API，你是开发者，具体应用怎么设计，不该事由我们批准。理想情况下，复杂任务会被拆成许多较小的调用；作为底层服务提供方，我们甚至不应该知道下游用户的完整任务是什么。这种边界能给软件工程师更大的自主空间。

我希望他们把能力用于好的事情。我们也讨论过开源、公益支持等办法，只是眼下还没精力落实。但只要由我负责，我就不希望把这些价值偏好直接固化进模型的底层判断里。

Swyx：既然说到平台与开发者的关系，也想澄清一下你们的隐私条款和使用条款。之前有些人产生了误解，觉得你们对 API 用途限制得很严。

Diogo Almeida：我不确定你具体指哪一条，不过我看到过一些关于基准测试的讨论。

Swyx：对。你之前公开说过，相关限制是预览期条款里留下的，正式发布时没有及时移除，团队准备修改。

Diogo Almeida：团队有些进展我甚至还没跟上。我请他们和律师确认这件事。我们显然不打算阻止用户测试和比较模型；恰恰相反，我支持他们这么做。

但这和我怎么看公开基准榜单是两回事。我非常不喜欢公开榜单。至于用某些内部基准作为实际能力的替代指标，我的态度也比较审慎。

Swyx：你担心的是榜单很快饱和，还是模型可能见过公开测试题，因此容易刷分？



Diogo Almeida：刷分容易，是原因之一。假设两年后，市场上有很多公司在做与 Jev 类似的产品，我们提供的价值可以说是“每美元获得多少智能”，或者“每秒获得多少智能”。大家很容易盯着价格和速度，因为它们好测量。但价格和等待时间是用户付出的成本，用户真正想得到的是有用的智能。

难就难在，模型到底有多好，有些部分很难用一个分数概括。Jev 发布后，开发者亲自试用产生的反应，甚至比发布视频更让我在意。他们感受到模型在实际使用中的表现，也感受到我们为此下了多少功夫。

公开基准想用数字帮助大家建立信任，但它太容易被针对性优化。即使团队主观上不想刷榜，也可能不断收集与测试集相似的数据，让模型在榜单上表现更好。过去有些实验室收集类似 MMLU 的题目来训练，在我看来，这不过是绕了几步优化基准成绩。

所以我认为，开发者可以先凭实际体验建立初步判断，随后必须把模型放进自己的工作流，测量它在那个具体任务上的表现。我们的工作则是持续提高可靠性，让它越来越值得信任。可靠性始终是 TypeSafe 要解决的问题；如果只想尽早发布一个能力不够好的版本，我们一年半前就可以推出 Jev 了。

4“最苦涩的教训”：先选对任务，再谈模型和数据

Swyx：你前面提到自己的“最苦涩的教训”(The Bitterest Lesson)，我们来展开讲讲。

Diogo Almeida：Sutton 的“苦涩的教训”强调算法和计算的重要性。但在我看来，数据比算力重要得多；而最难、也最重要的，是先选对任务，找到研究的 North Star（核心目标）。

大语言模型的发展中，研究目标发生过几次这样的转变。RLHF 把重点转向指令遵循，当时没人意识到模型能在这件事上做到那种程度。RLVR 又把方向往前推了一步。现在，我们用 RLCD 定义了另一个任务：让程序能够把模型放进运行流程，可靠地调用它的判断。数据对此至关重要。我怎么强调都不为过。

Swyx：所以你们更愿意把 TypeSafe 称为一家“数据实验室”，而不是“模型实验室”？

Diogo Almeida：完全正确。我们会一直非常重视数据。对我来说，模型能力的提升，很大程度上就是数据问题。数据工作极其复杂，却也能把可靠性一位一位地往上推。数据选取和处理方式稍有变化，结果就可能大不一样。

Swyx：你们也在大量招聘数据人才。怎样才算优秀的数据工作者？你说过你们使用合成数据，但“数据是合成的”显然只说到了表面。是不是还要有人认真检查这些数据，指出问题，再回去重新生成？

Diogo Almeida：是，但完整回答会复杂得多。我给新入职的数据同事做的培训，可能比这期播客还长。这里先讲最重要的几层意思。

首先，任务决定数据的形态。RLVR 需要的数据在某种程度上是供模型探索的环境；RLHF 需要人类反馈。每种任务都有自己的数据形式，我们也一样。所以我们做的合成数据，并不是一个通用模板生成出来的东西。

其次，我们不想用用户数据训练模型，即使取得许可、技术上也做得到，我们仍不想依赖它。真实世界的数据有很强的偏向：很多人反复提出相似问题，需求呈现幂律分布。直接按这些数据训练，模型容易对高频场景过拟合，其他能力反而变得不均匀。



我们瞄准的是多年后的应用。到那时，模型也许会成为深入软件技术栈各层的通用基础设施，支撑今天还没人想到的产品。如果打个比方，可以把一般的大语言模型想成 UDP，把我们的模型想成 TCP：开发者能在上面构建各式各样的应用。即使我们拿到了今天所有用户数据，它也只能让模型更适应今天，未必能帮助开发者做出未来的软件。

因此，数据团队的工作有点像艺术创作。他们研究模型的“认知核心”，找出能力不平整、不稳定的地方，再有针对性地处理。我们认为自己的模型在这方面已经相对平滑，但不可能做到完美。一次修正最好能改善一类问题，在过去、现在和未来不同情境下都有效，而不只是修好某一道题。

Swyx：解决普遍问题，而不是只解决一个具体案例。

Diogo Almeida：正是这样。每次做到这一点，都需要很强的判断力。

5“Jev 出现之前的 AI 世界，是个悲剧”

Swyx：你刚才说，我理解数据的方式受 RLVR 影响很深，这个批评挺公允的。那我们具体聊聊 RLCD。你们目前还没有发表介绍它的论文，对吗？在不了解技术细节的情况下，大家怎样判断 RLCD 确实代表一种不同的方向，而不只是一个新造的术语？“校准”这个概念我能理解，但 RLCD 与大家熟悉的 RLHF 究竟有什么区别？

Diogo Almeida：还没有发表论文。你的问题很好，不过要回答它，我想先反问一句：RLHF 指的是什么？

这个词在不同时期指过好几种工作。最早有通过人类偏好反馈，让系统学会完成难以精确规定的任务的研究。我记得有一项工作涉及教机器人做后空翻，可能与 Paul Christiano 等人的研究有关，但具体是哪篇，我也不能完全确定。PPO 是一种算法，并不天然意味着奖励一定来自人类反馈。

后来 OpenAI 做了“学习摘要”的工作，把 PPO 用在语言模型上，让模型学习完成摘要这类很难用简单规则规定好坏的任务。人们也会把这称为 RLHF，不过我没有参与那篇论文。

但当我说 RLHF 时，我更想强调的是指令遵循这个任务，而不是某篇论文采用了 PPO。真正重要的，是研究者确定了一个新的 North Star：让模型理解并遵循人的指令，是一个值得持续优化的方向。今天的 DPO 及其后续方法即使没有使用那篇论文里的算法，也仍是在做这个意义上的 RLHF。

同样，RLCD 对我们来说是在提出一个新的任务目标。我使用这个名字，是想准确说明我们在优化什么：让 AI 能被程序调用，成为软件可以使用的能力。

Swyx：也就是说，关键词是“可编程的 AI”：让程序能够在不需要人每一步介入的情况下使用模型，进而自动化更多工作。我理解你的 North Star 对吗？

Diogo Almeida：对，但还要加一个前提：必须务实。我们得看清当前语言模型实际擅长什么。有些面向程序的新输出类型在概念上可能很棒；如果技术还不足以让它可靠工作，暂时不推出，也没什么可遗憾的。

在我看来，Jev 出现之前的 AI 世界，简直是个悲剧。这话听起来很狂妄，但这个想法在我创办公司很久以前就有了。



Swyx：这点我可以作证。你已经谈了好几年。

Diogo Almeida：是。我起初以为做出一个能供程序调用的模型不会太难，整个项目一周就能完成。事实证明我错得离谱。以前我甚至觉得：“这个问题我马上就能解决。”现在想来，我得向当时被我低估的 OpenAI 同事道歉。

AI 领域还有另一件令人遗憾的事：承诺很大，实际交付却跟不上。我认为 RLHF 和 RLVR 路线都存在这个问题。但让我一直放不下的，是 AI 明明已经表现出很强的能力，却仍有大量简单工作无法自动化。

我在演讲时常问：为什么 AI 能处理极其困难的数学问题，却连许多基础、重复的工作都接不过去？这些工作不要求人有多高的智力，也谈不上令人满足，但现实中仍得有人做，因为软件还没法可靠地把它们自动化。我们已经有了——一台很强的“智能引擎”，却缺少合适的接口，把它接进那些有经济价值的流程。

即使有一天 TypeSafe 不复存在，这个方向也已经被打开。别人也许需要一两年追上来，也可能更快；如果模型质量的差异持续重要，我们会有更长时间的优势。但比公司竞争更重要的是，整个领域开始探索：智能还可以怎样接入软件。我相信这会改变技术接下来的发展路径。

Swyx：我完全同意。你们真正创造的是一种新的可能性。我试着替你概括一下，方便大家理解：不要把 TypeSafe 和 Jev 的成功看成仅仅是“出现了一种新的模型类型，我可能可以继续沿用之前的路径去做事”。不是这样，实际上，可能还有好几种别的模型类型值得探索，这个行业应当是百花齐放的态势，应该让各种尝试奋都发展家里。其中有些方向，你们大概也会亲自去做。

Diogo Almeida：完全正确。那种早期互联网的活力又回来了。我觉得我们重新回到了某种技术乌托邦时刻，再也不用像以前那样感慨：“唉，有时候我的编程智能体能工作，但真正最好的东西都被公司内部藏起来了”。创造新东西又成了一种真实的可能。当然，接下来会是一个相当疯狂的世界，大家最好坐稳了。我对此兴奋极了。

Swyx：现在你们有了资金，也有了发布后的势头，终于能把想法做出来给大家看。

Diogo Almeida：对。以前一直讲，却不能把最关键的东西拿出来，感觉像总在吊大家胃口。我的演讲尤其如此：我说 AI 应该推动自动化，却没法展示它具体怎样做到。你之前看过我们的宣言，还说有些地方写得太模糊：“第一步是什么？你们说的智能模型究竟是什么？”

Swyx：我当时问你模型在哪里，你只说“快有了”。我主要对“可组合”这个词有意见，不过“Build Prod, Not God”这句口号很好。

Diogo Almeida：谢谢。团队很认同这句口号。我们对正在做的事很兴奋，但大家也很务实。

Swyx：宣言里还有一份“秘密总体计划”，讲怎样一步步构建面向机器、可组合的 AI。

Diogo Almeida：写成“秘密总体计划”是你的建议，我得正式把功劳记给你。

Swyx：谢谢。不过你当时只给我看了一半故事，没有告诉我还要发布模型。那时候也没有《Doom》演示，没有性能数字，我自然会问：你们到底打算拿什么证明它？

Diogo Almeida：问题是，我不相信公开基准榜单能证明这件事。模型好不好，最终得让人放进实际任务里体验。这种做法有助于建立长期信任，但也让我们吃过苦头。去年融资时，很多人



不相信我们，只想看基准测试成绩。我们不愿意为了拿出好看的数字，去迎合一套容易奖励刷榜行为的评价方式。

Swyx：选择这条更难的路，最后也让你建成了一家自己愿意待的公司。

Diogo Almeida：是。我不太后悔这个选择。昨晚聊到为什么离开 OpenAI，我还挺激动的。发布前，我常这样想：如果 AI 最终因为承诺过多、实际交付不足而进入新一轮低谷，而我没有尽力探索另一条路，我会觉得自己也有责任。一方面，我参与过 RLHF 这条路线；另一方面，我相信让 AI 真正进入软件自动化，能创造很大的价值。

发布后，我对这件事的说法有些变化。至少在我看来，自己担心的那种“AI 寒冬”已经不那么不可避免了。Jev 上线还不到一周，我已经看到开发者把它用在真实工作中。眼下就像西部拓荒时期一样，充满了未知，各种尝试都在冒出来。

6 爆红之后，Jev 更在意什么

Swyx：能不能分享一些发布之后的具体数据？例如注册的用户数量之类的。

Diogo Almeida：具体数字主要是团队在跟进，而且每天都在变。不过有一个里程碑我记得很清楚：日调用量已经超过一万亿 Token。这不是发布当天大家尝鲜带来的短暂高峰。夜里调用也没有停下来，说明有程序在持续使用 Jev，而不只是人在界面里试几次。这让我特别兴奋。

相比之下，我没那么在意注册人数。我们发布时没有专职营销人员，大家看到的宣传基本就是团队自己的表达。候补名单一度增长很快，我们也在不断放人进来。平台团队承受住了这次发布带来的流量，做得非常出色。

但我们后来意识到：对开发者平台来说，候补名单上的人数并不能说明多少问题。其中可能有不少人不是开发者。他们进来试了几个问题，发现 Jev 不是聊天机器人，就会疑惑：“我的下一个 ChatGPT 在哪里？”我还没有精确计算过，不过我的直觉是：哪怕全世界每个人都来试几次，调用量可能还不如一个真正用它创造价值的开发者写下的一段循环程序。

我们起初以为，放多少人进入候补名单需要非常谨慎。后来发现，更紧张的是速率限制：开发者一旦把 Jev 用进程序，获得了实际价值，就会希望大幅提高调用额度。软件就是这样运行的——先花时间定义一个重复任务，之后让它持续执行；只要创造的价值超过最初投入，就可以把它放到后台，再让其他功能依赖它。

Swyx：设置好，让它自己运行。

Diogo Almeida：对。之后开发者还能在这些能力之上构建更复杂的东西。许多早期互联网的创造，正是通过不同软件能力的组合才发生的。虽然你不喜欢“可组合”这个词，我仍想在宣言里强调它：我们希望 Jev 成为催化剂，让开发者做出连我们也没预料到的应用。为此，我愿意继续在 Discord、公开交流和播客里跟他们沟通，哪怕要在直播里套个垃圾袋出镜。

Swyx：长访谈也有这个好处。我们可以越过发布初期比较表面的讨论，让人理解你们究竟想做什么。认同这个方向的人，也许会加入团队，或者“买下你们”——抱歉，我是说成为客户。

Diogo Almeida：哦，作为客户（笑）。

Swyx：对，作为客户。发布视频也确实很受关注，刚才看到是 3600 万次播放。



Diogo Almeida：现在到 3800 万了。

Swyx：如果只看 2026 年新兴 AI 实验室的发布声量，你们可能排在最前面。

Diogo Almeida：谢谢，不过我不太在意“新兴 AI 实验室”这个标签。我们还有周边专门拿它开玩笑，比如“你最喜欢的新兴实验室最喜欢的新兴实验室”，以及“有产品的新兴实验室”。

我真正想做的是一个可靠的开发者平台。希望发布热度过去之后，开发者仍能长期依赖它、信任它。

7 模型不能说变就变：Jev 如何兑现“可靠”？

Swyx：你们一直强调可靠性。我原以为主要指 RLCD 里的概率校准，但看起来，它也包括服务可用性、扩展能力，以及常说的“几个 9”，对吗？

Diogo Almeida：那些都是可靠性的一部分。但模型的判断本身也要可靠：它能不能持续完成你希望它做的事？现在的大型推理模型很聪明，可在我看来，仍有不少任务是它们看起来足够聪明、企业也有动力自动化，最后却不敢交给它们做，因为结果还不够稳定。我们离理想状态也有很长的路要走。

我想先把容易的工作自动化，再去处理难的工作。我的目标是，有一天开发者需要一处智能判断时，可以像写数据库查询一样，直接在程序里写一个类型安全的 System 1 调用，让程序准确地走向相应分支，而不必每次都先试探模型能否胜任。这会是一个漫长的过程。

Swyx：说到可靠性，我注意到 API 里没有随机种子（seed）参数。同样的输入，每次都会得到同样的输出吗？如果不会，为什么？

Diogo Almeida：这是个好问题。“可靠性”其实是一个总称。AI 无法自动化某件事，可能是输出不符合类型要求，可能是结果不确定，也可能是能力分布不均匀。你问的确定性，指相同输入得到相同输出；它对单元测试等场景可能有价值，但我不认为它应该是首要目标。

我更重视稳健性：语义相近的输入，应该得到相近的判断。大语言模型在这一点上有时很不可靠。我们的一个测试办法，是在提示词里加入不同的 UUID 作为随机标记。问题的实际含义没有变，只是多了一个无关的字符串；模型的判断就不该因此大幅变化。开发者在让 AI 做决定时，常常会被这种细微变化导致的结果波动坑到。

当然，我们也可以做确定性模型。如果开发者能告诉我们它在哪些场景确实重要，我们愿意考虑。但确定性可能需要在成本与能力之间作取舍。我们一直努力站在“每美元智能”的帕累托前沿，为此尝试了很多不那么常规的模型组合与工程方法。

Swyx：所以，回到最初的问题：将来会不会提供随机种子或确定性模式？

Diogo Almeida：有可能。只要需求足够明确，而且我们没有被 GPU 资源严重卡住，就可以做。只是按我们目前的判断，它可能降低每美元能提供的有效智能。

Swyx：从 OpenAI 和 Anthropic 的发展过程看，我猜开发者迟早会要求这个功能。即使你告诉他们稳健性更重要，他们可能还是会想要确定性。

Diogo Almeida：那就看后面会怎样吧。有人说我们品牌的一部分是“立场坚定”，其实就是委婉地说我固执。



Swyx：但我以前和你争论过。证据充分时，你也会改变原先的判断。

Diogo Almeida：是，你在开发者需求方面说过很多次。这一点我认。不过，我怀疑我们会在很长一段时间里受到 GPU 供给限制。如果一种方案提供同等智能却消耗更多 GPU，我们就更难让更多人用上它。我们当然希望服务企业，但眼下更想让尽可能多的开发者拿到 Jev，去实验、去做那些出乎意料的东西，让这个方向真正热闹起来。

Swyx：这种探索已经开始了。不过我想提醒你一个开发者可能会担心的问题：你说会尽一切办法提高“每美元智能”，又面临 GPU 限制，大家可能会猜，你们会不会在发布后把模型进一步量化，以节省显存或带宽，却让同一个模型的质量下降？

你现在不一定要作承诺，但至少应该考虑明确告诉开发者：已经发布的模型，其质量和行为不会保持稳定。过去有些 API 虽然模型名称没变，背后的模型却变了。你们现在有版本号，这是好事；还可以进一步说明，同一个版本上线后是否会保持不变。

Diogo Almeida：已经部署的模型，我们不会悄悄改动。对开发者来说，那太离谱了。直接面向用户的产品可以持续调整背后的模型，只要产品体验变好就行；但 API 是别人程序中的依赖，不能按同样的方式处理。

不过，我们更新模型的速度可能会比大家习惯的快得多。我们会不断发布新版本，也暂时不能承诺每个旧版本都有长期支持，因为还有很多改进要做。眼下有不少人在使用 Jev 1.13.0，我们可能会暂时把它作为长期支持版本；我知道开发者不愿意看到依赖突然失效。

另一方面，如果每发布一个版本就永久保留，推理服务可能要同时运行大量不同模型，这也会给所有人带来负担。

Swyx：更新快的话，可能很快就有上百个版本要维护。

Diogo Almeida：正是如此。我们希望将来找到比简单保留某个长期支持版本更好的办法，团队正在研究。我认为那会对开发者很有帮助，但现在还不能把尚未实现的方案当成承诺。

可以确定的是，我们会继续推出能力更强的新模型。按我的观察，Jev 在相邻版本之间的行为差异，通常甚至小于一些生成文本的模型对同一个请求调用两次产生的差异。但如果我们解决了模型能力中某个明显不稳定的部分，新旧版本之间就可能出现很大的变化。

8 不追 Benchmark，追极致的“每美元智能”

Swyx：模型有了长期支持版本（LTS），还有一个好处：你们可以把同一个版本移植到其他芯片平台上运行。不知道你们有没有考虑过这件事。

Diogo Almeida：暂时不评论。我更关心的是“每美元能获得多少智能”。

Swyx：但速度也重要。

Diogo Almeida：这要看后续情况。

Swyx：过去一年，推理技术路线发展得很快。比如，把模型放到 Cerebras、Etched 这类芯片上运行，速度可能会有大幅提升。



Diogo Almeida：对。“每秒能获得多少智能”是另一个衡量指标。我们内部甚至讨论过把成本和时间放在一起衡量，比如“每美元、每秒能获得多少智能”。

对于 Jev 系列模型，我在内部反复强调的一点是：模型变得更聪明当然好，但它必须处在帕累托前沿。也就是说，在同等成本下，它要尽可能强；达到同等能力，成本要尽可能低。这就是 Jev 的定位：把每美元获得的智能做到最好。

至于每秒能获得多少智能，还要继续看。这个方向很有意思，我也知道有不少行业高度依赖实时响应。对它们来说，速度的提升直接对应着很大的经济价值。我当然希望两个指标都能做好，也希望从市场和用户的反馈中判断，究竟该把重心放在哪里。

Swyx：而且速度不只关系到实时场景，也关系到规模。到了足够大的调用量，哪怕每次只差几微秒，乘上数十亿、数万亿次，也会变得很可观。

Diogo Almeida：如果是大型数据库 MapReduce 查询这类后台任务，延迟可能没有成本那么重要。这也是 Jev 的定位：尽可能提高每美元所能获得的智能。至于“每秒智能”，还要继续看。我很喜欢那些对速度要求极高的应用场景，也知道对一些高度依赖实时响应的行业来说，速度提升能带来很大的经济价值。这方面的需求正在增长，很有意思，但我不认为它会成为 Jev 的主要方向。

Swyx：说到“更快、更便宜”，其他模型通常给出的取舍是速度更快，但价格也更高。我在想，Jev 之所以受到关注，一个原因是你们做到了更快、同时更便宜。这一块目前选择不多，前提当然是模型的智能水平大致保持不变。

Diogo Almeida：“智能水平保持不变”这句话很关键。难就难在这里。

Swyx：但你似乎不太认可公开基准测试，或者至少不喜欢用现有的公开基准来证明这一点。你们内部总得有一套判断方法吧？

Diogo Almeida：当然。我们有自己的内部评测。不过，要避免团队有意无意地针对评测做优化，需要很强的自律；这件事必须得到最高优先级的重视。

否则，我们凭什么说自己的模型处于“每美元智能”的帕累托前沿？我们不是凭感觉做决策。尝试不同方案时，成本和能力都会变，我们会测量、比较，把结果放在一起看，判断哪种方案对用户最好。

所以，我并不反对评测。我担心的是，一旦掺入其他激励，评测就可能变得很危险。在这件事上我管得很严。同事们也许会觉得我在很多事情上都管得严，但对我来说，不能在模型到底有多聪明这件事上自欺欺人，这一点尤其重要。我们得诚实地追求事实。

Swyx：同意。接下来我想聊聊 API 设计上的一些具体选择。

9 别让 AI 一口气干完：把任务拆小，出错才好查

Swyx：你们定义了三个基础概念：Choice、Score 和 Noul（又名 Noulli）。先说 Noul，这个词是从哪里来的？是文献里已有的术语吗？

Diogo Almeida：现在算是了（笑）。我们为名字讨论了很久。它有点像布尔值（Bool）：涉及“真”或“假”，但它的取值是连续的。名字来自伯努利（Bernoulli），更准确地说，是取了 Bernoulli 的一部分，因为它表达的就是伯努利概率。



我们当时还考虑过 PBool、Pool 等名字。我甚至想把它叫作“pool party”，但没人同意。最后大家觉得 Noul 最合适。我们这群人起名多少有点不按常理出牌，Jev 也是这样。最初我们以为，对程序员来说，Jev 不过是代码里的一个字符串，没想到这个名字后来还衍生出不少双关梗。当时内部有很多人反对这个名字，现在除了一个人，其他人都向我道过歉了。

回到 Noul。我们需要为它创造一个新概念，因为如果直接叫 Bool，会让人误解。事实上，Choice、Score 和 Noul 都是我们有意单独定义的概念。它们与编程语言中的现有类型很接近，但并不完全相同。比如 Score 不是整数。如果通过 Instructor、Pydantic 之类的工具，直接把整数或浮点数映射成 Score，就可能出问题。我们更倾向于把含义说清楚，即使这会增加一些初次理解的成本。

Swyx：但你们不担心集成问题吗？开发者通常希望它能直接接入自己已经在用的工具。你们有自己的 SDK；如果我是开发者关系负责人，我可能还会特别在意：怎样把 Jev 和 Instructor 一起用？怎样接入其他工具？

Diogo Almeida：我们确实希望做好这些集成。相关示例也许已经有了，只是我现在跟不上所有更新。社区里也会有人做。支持社区这件事没有一个“完成”与“没完成”的二元状态，我们总还能做得更好。说实话，聊到文档我有点紧张，因为录制前没来得及重新看，而文档又一直在变。

至于这三个概念，Score 其实并不难理解。它接近用语言模型做评判（LM judging）时得到的评分，也可以把它叫作一种“判断结果”。Noul 也可以被看作概率，但在我们的体系里，模型输出本来就带有概率性质。Choice 则最接近函数调用，不过我对现有的函数调用设计有不少意见，这个话题我们可以稍后再展开。在代码中，Choice 更像是把 switch 或 match 语句要处理的选择，直接作为 API 输出。

Swyx：也就是说，它可以很自然地对应枚举（enum）；如果需要，开发者再根据选中的项调用相应函数。

Diogo Almeida：对。关键是先得到那个“选择”。这三个概念都可以对应到基本的程序控制方式：Choice 对应基于枚举的 switch 分支，Noul 对应 if 条件判断，Score 对应排序，或者通过“大于”“小于”的阈值来筛选。这一直是我们的设计方向。以后还会有更多类型，也会继续与程序中的基本操作对应起来。

Swyx：对于准备深入使用 Jev 的开发者，还有什么 API 设计上的细节值得讲？比如这些概念具体怎么用，图例、置信度该怎么看，测试中又该注意什么。你是最了解它的人，我想多给他们一些实际建议。

Diogo Almeida：谢谢，我很喜欢这个问题。没想到会聊得这么细。除了几个月前给我们的开发者关系同事做入职培训，已经很久没人这样问我了。

我们设计模型时，想的是让它将来能深入计算机程序内部工作。我们相信，这类用途的规模会远超今天人们的想象。模型现在也许还没准备好，但我们一直在朝这个方向推进。我们的目标不只是让它在相对浅层的任务上不断提高分数，而是让它进入程序的核心逻辑，因为这样才能让软件具备更强的能力。

刚才说的 Choice、Score 和 Noul 是输出形式。输入端同样可以有结构：状态（state）、指令（instructions）、判断标准（criteria），都可以是结构化的 JSON 对象。程序可以把这些信息准确地放到需要的位置，开发者不必先把它们塞进字符串模板。



很多人可能没注意到这一点，以为这些输入都只是字符串。用模板拼出一条系统消息，当然也能工作，但那延续的是过去处理模型输入的方式。既然信息本身有结构，就应该尽量保留这种结构，让计算机直接处理。写普通程序时，我们不会先把所有数字转成字符串，再传给程序内部的其他部分。转成字符串通常是为了展示给人看；程序内部传递的，应当是有明确含义的嵌套结构。

我们也在持续优化模型处理这类输入的能力。它现在已经能处理结构化信息，但嵌套每增加一层，推理难度也会提高，所以这仍是我们重点投入的方向。我也希望开发者更多地采用这种方式：代码会更清楚，也不必过度依赖底层实现细节。

可以把它想成调用一个 AI 函数：程序当前有一组状态，也就是可用的变量；你需要决定，哪些状态应该传给这个函数。相比之下，一条包罗万象的系统消息很像一个混乱的全局变量：把所有信息、所有指令都放进去，然后指望模型一次满足每项要求。很多问题其实可以拆开，并行地问。

我很建议大家多向模型提问，把问题拆小、拆细。我这么说不是为了增加调用量。无论当前模型能否一次完成复杂任务，把问题分解成简单决策，都会让 AI 应用的代码更好维护；而且每个小决策都更容易单独评估。

过去我们常常给模型一大段系统消息，等它返回一大段结果，再检查它有没有遵守其中的每条要求。现在回头看，这种做法挺不可思议的，只是大家已经习惯了。比如“不要读取这个子目录”“不要把 API 密钥传给 DeepSeek”，这类约束应该尽可能由程序保证。机器学习模型本身无法给你绝对保证，但把任务拆开之后，至少可以逐项测量、验证。

Swyx：比如可以核查某个操作到底有没有执行。

Diogo Almeida：对。我们的接口本身就很容易验证，这会让工程实现可靠得多。

Swyx：我理解这个思路。不过，以前大家不这么做，也有现实原因：即使用小模型，把任务拆成很多次调用，仍然可能又慢又贵。我自己做过对比：一条流程把所有要求放进系统消息，只调用一次、拿到一个完整输出；另一条流程把任务拆成上百个问题。后者更慢、更贵，结果还不如前者。

Diogo Almeida：确实会发生这种情况。拆分任务也很麻烦，调用之间可能还得重复传入一些信息，看起来不够高效。于是人很容易想：为什么不一次性把所有东西都放进去？但那样得到的系统往往很难依赖。我希望我们的模型能够支持大量在后台运行的程序调用；如果做不到，我会很失望。

Swyx：那具体应该怎么拆？你们有没有发现什么有效的方法，或者发现哪些原以为可行的拆法其实不行？开发者听完之后可能会照着这个思路使用 Jev，但他们首先得知道从哪里下手。

Diogo Almeida：我倾向于把问题拆到最小的语义单元：先找到其中最基础、能够独立判断的那件事。我可能是使用我们模型最多的人之一。提问时，我会尽量把输入写得结构化、明确；在问题里清楚指出所引用的内容，有时会用反引号把它标出来。我希望模型尽可能按字面准确理解要求。编程本来就要求指令得到准确执行，而 AI 扩大了程序所能执行的指令范围。

我自己偶尔也会图省事，把几个判断混在一个问题里。但如果是大型生产系统，我会不断增加独立的问题，并且让新增问题这件事足够容易。每个问题负责什么，要拆得清楚；最终的具体行为则由代码来控制。



举个小例子：拒绝回答。我不建议只问模型一句“这里要不要拒绝？”这个问题它或许也能答得不错，因为这是一个适合快速直觉判断的任务。但更好的办法是，针对不同的拒绝情形分别提出独立的问题。这样，你就能明确规定自己希望系统如何处理各种情况，而不是让模型自行猜测。

更妙的是，如果后来发现某种情况没有触发拒绝，原因是你漏写了一项条件，这其实是一个很好的工程问题：补上那个问题，设置阈值，再把这个案例留作测试。修正就留在程序里了，不会因为提示词变长、上下文中的早期信息失效而被“忘掉”；你也可以持续测量它。如果模型对某项判断还不够准，还能根据真实案例调整相应阈值。整个过程有点像在做机器学习系统，但你不需要为每种情况重新训练模型。

当然，有些任务可能仍超出了模型当前的能力。比如看到有人用模型做自动交易，我会有点担心。这件事看上去很酷，但任务本身复杂、风险也高，最好交给专业人士。即使把它拆成若干判断，你也可能通过评测发现：模型在某一步还不够可靠，那这一版就先不要部署，或者调整方案、选择更保守的处理方式。

客户服务也一样。假设模型还不能可靠识别“VIP 客户在说反话”这类特殊情形，你就可以规定：遇到这种情况，转交人工处理。置信度估计也是为这样的决策服务的。

Swyx：你刚才提到，开发者可以通过设置阈值，决定什么情况下采用模型的判断。但如果模型的概率校准本身就有问题呢？一个校准良好的模型，至少应该让较低的分数的对应较低的实际发生概率，较高的分数对应较高的概率。现实中，两者也可能对不上。这时我可能想微调模型，而你们目前没有提供这个功能。

Diogo Almeida：我没有说模型的校准是完美的。微调当然是我们可以考虑的方向。

Swyx：微调是一种可能，也可以是别的调节手段。现在听起来，如果模型判断错了，用户能做的就是改提示词、把问题拆得更细，或者调整置信度阈值。如果模型确实不擅长这件事，这些办法不一定让人满意。

Diogo Almeida：没错。先说清楚，模型会在很多事情上犯错。我们有问题反馈按钮，大家也可以在 Discord 告诉我们哪里出了问题。我们希望持续改进模型，每个新版本都应该有明显提升；如果没有足够大的进步，我们就不会保持现在这样的发布速度。

承认 AI 在某些任务上还不够好，是务实的做法。同时，“足够好”也取决于具体用途。人也会把工作做错，但只要总体预期收益足够高，很多工作仍值得由人来做。模型也类似：通过合适的阈值和其他控制方式，即便它会出错，仍可能承担相当多的工作。

至于微调，我能想象我们以后会提供，但我有一些顾虑。用户想要的功能和真正对他们有益的功能，不一定完全相同。通用模型有一种很难说清的优势：它会处理大量其他任务，这种广泛的能力，可能也让它在某个特定任务的边缘案例上表现更好。如果只针对一个狭窄任务做微调，我会担心失去这部分能力。

所以我的回答是：有可能。我在这些事情上很务实，我们想做的东西还有很多。但我也也不想推出一个看似有用、实际却很容易让用户把系统用坏的功能。

Swyx：OpenAI、Claude，可能还有 Gemini，都曾推出过微调功能，后来又撤回或收缩了相关服务。这也挺有意思：现在微调似乎更多发生在开源模型领域。

Diogo Almeida：我知道这些情况。有些微调产品的效果确实不好，撤下来也许是对的。



Swyx：所以，告诉用户“微调可能不是正确的解决办法”，也说得通。另一种回答是：你们的模型与通常的大语言模型差异很大，就像量化、输出 Token 这些概念未必适用于它，微调也可能不适用。

Diogo Almeida：我对这种可能性也持开放态度。不过下面说的是我的设想，不是产品承诺。

随着“每美元智能”的成本继续下降，也许我们能用很小、成本很低的模型，完成一些过去要靠手写规则处理的判断。举个例子：会不会有一天，人们不再需要自己写正则表达式？因为用 AI 完成同一件事的成本，已经低于编写和维护正则表达式所带来的复杂度。我会很乐意看到那一天。对于其中一些狭窄用途，微调也许恰好能让模型的表现跨过可用的门槛，还得继续看。

我希望概率校准加上模型级联，也能解决一部分问题。比如，小模型对某个判断非常有把握，就直接采用它的结果；如果置信度落在中间地带，再交给更大的模型，必要时继续往上一级处理。我还不知道最终哪种方式效果最好。

还可以设想另一种用法：我们提供一系列处于帕累托前沿、规模各不相同的模型。熟悉技术的开发者也许愿意自己选择，但企业可能更需要动态调度：系统根据技术栈中不同环节所需的判断能力，自动选择合适的模型。考虑到我们的接口相对简单，这套机制甚至可能结合某种自动微调。

重申一下，刚才说的动态选模型、自动微调，都不是产品承诺。我只是在畅想一种有点像科幻的未来。

10“决策模型早就有了”，Jev 到底想做什么？

Swyx：但你们会考虑推出不同规模的 Jev 模型，让用户有得选？

Diogo Almeida：当然。用户到底需要多强的智能，我怎么可能事先知道？

Swyx：需求可能是无限的。

Diogo Almeida：也有人劝我们，现在的产品已经够好了，不用急着发布新东西。但我不太喜欢“够用就停”的想法。有句话我很认同，大意是：市场不奖励你做一件事时，你仍然坚持去做，那才体现出你的文化。我希望大家以后也拿这句话要求我，因为说出口之后，就很难反悔了。

也许有一天，我们坚持的东西会变得理所当然，公司会像 Visa 一样，成为人们平时不会特别留意的基础服务，到时候我可能连粉色西装都不穿了。但现在，我还想让更多人看到这条路的可能性。我们会继续做有意思的事，并不是因为当前产品非改不可，而是因为今天大家看到的才刚刚开始。第一次发布更像是一次低调的研究预览，远不是我们能做的全部。面向机器的智能（machine-native intelligence）还有很大的空间。

Swyx：所以，将来可能不止一种模型规模，也不止目前这一款模型？

Diogo Almeida：没错。我们希望尽可能满足不同需求。但我得加一个重要前提：我不想像一些产品团队那样，什么想法都往外推。我们做的东西应该服从一个统一的方向。回看我们的宣言，未来的尝试都应该落在其中三个方向之一。

Swyx：我没准备好在这里逐条聊那份宣言。



Diogo Almeida：抱歉，我说得像是在故意吊胃口。我的意思是，那三个方向不是一张待完成的清单，而是我们认为足以支撑下一轮技术变革的三条轴线。我们做的每一次尝试，都应该能在其中找到位置。模型方面，我们也会做一些看起来挺奇怪的东西。

既然是“面向机器”的智能，就不一定非得让人一眼看懂它的形式；关键是它有没有实际价值。

Swyx：能透露一点吗？“奇怪”会是什么样？

Diogo Almeida：给个提示吧。有人想把我们的模型叫作“决策模型”，因为目前的几个基础概念都与决策有关。但我不会这样命名：未来还可能出现其他面向机器的输出类型，它们未必是决策。

Swyx：好，那就留给大家猜。

Diogo Almeida：这个提示已经挺有意思了。

Swyx：现在也有人说：“决策模型我一年前就做过了，Jev 没什么新鲜的。”但我觉得，一方面，你们展示了这类模型可以怎样成为一个独立的产品方向；另一方面，从我看到的数字和测试结果来看，Jev 目前的表现仍然超过了那些类似产品。

Diogo Almeida：先说清楚，我不在乎基准测试上的输赢。无论领先还是落后，我都希望继续发布我们的进展。

Swyx：你们至少让更多人开始认真看待这个方向。而且，“决策模型”和 System 1 模型之间的区别，似乎也是你一直想讲清楚的。

Diogo Almeida：对，我想让软件工程师借助 AI，拥有更强的能力。让我难过的是，AI 已经这么强大，却远没有得到充分利用。如果事情继续这样发展，甚至会走向新一轮“AI 寒冬”，实在太可惜了。这件事会让我情绪激动。

我并不是认为所有技术进步天然都是好事，也不想一味宣扬技术乐观主义。但看到 AI 有这么多能力，软件却仍然没有真正用上它，我觉得很难接受。我想做的，就是帮大家打开这些可能性。先说到这里吧，这几天我已经哭了太多次，不想在录节目时再哭一次。

Swyx：谢谢你愿意讲这些。光看“TypeSafe AI”这个名字，大家未必能感受到你对这件事的投入。但了解你们想推动的方向，以及你们已经迈出的第一步之后，就更容易理解你为什么希望大家一起往前走。

Diogo Almeida：我不确定最难的部分是否已经过去。以后肯定还有很多难题。也许等到各种工作真正实现自动化、经济增长明显加快，每天都有值得庆祝的进展，我们才能说难关已经过去。而且，我觉得大家现在太关注速度和成本，对可靠性的关注还不够。可靠，才会让人用起来觉得好；可靠，才会让人敢于信任它。

Swyx：你们还写过一个目标：五年内让全要素生产率（TFP）增长超过 3%。我很少见到一家 AI 实验室把 TFP 增长当成目标。

Diogo Almeida：因为那才是经济变革该有的样子。这和 OpenAI 早期章程想表达的方向其实相当一致。章程本身也许没变，但关于什么算 AGI，后来的讨论似乎越来越倾向于用利润之类的指标来界定。



在我看来，一个值得追问的问题是：如果模型能解出数学上的千禧年难题，却几乎没有承担现实世界中有经济价值的工作，我们该怎样评价它的影响？如果按“承担世界上大部分有经济价值的工作”这个标准看，我觉得目前各家模型都还接近零。这个过程也许已经开始，但我猜占比还不到 1%。等它真正发生时，应该能从经济统计数据里看到变化。

我期待那一天。我不认为它会造成大规模失业，但它会带来很多积极的变化。另一方面，我也厌倦了 AI 总是站在产品最显眼的位置。软件和生活应该变得更好，而 AI 只需要在其中发挥作用。

Swyx：让 AI 融入后台。

Diogo Almeida：对。我在演讲里也常问：2019 年的软件和 SaaS 产品已经创造了很大价值。现在都到 2026 年了，AI 的能力进步这么多，为什么大多数软件看上去却几乎没变？很多产品只是侧边栏多了一个聊天框。它有时能帮上一点忙，但公司仍不敢让它处理那些需要承担后果的决策，因为还无法充分信任它。

这在我看来很不可思议。企业有很强的经济动力去改变现状。我觉得接下来可能出现的，恰恰是所谓“SaaS 末日”的反面：SaaS 会因为 AI 而得到增强。软件公司最清楚自己的业务里哪些工作值得自动化，因此也最有机会把 AI 真正用进去。

Swyx：你们已经做出了一件很有意思的事。不过，我还想弄清一个边界：什么样的问题适合 System 1，什么样的问题需要 System 2？现在大家可能什么都想拿 Jev 试一遍，其中一些尝试恐怕会失败。

Diogo Almeida：“什么都拿 Jev 试一遍”这个说法挺好玩。说实话，边界要靠实验来找，就像缩放规律也来自经验观察。以机器人为例，投入了这么多钱，为什么它至今仍有很多事情做不好？问题未必能靠继续增加投入解决，还要看实验结果究竟能不能支持那条路线。

我的判断是，预训练模型把大量能力浓缩到了一起，而 System 1 最接近它们天然擅长的思考方式。可验证奖励强化学习（RLVR）在增强 System 2 式推理方面取得了惊人的进展，我很佩服这些成果。但这类能力也很脆弱。

回想 ChatGPT 刚出现时，人们常说：“它很通用，能做很多事”，接着又指出它不擅长数学，连 GSM8K 这类小学数学题也做不好。如今谈起经过 RLVR 训练的模型，大家又会说它的能力很不均匀：为什么它能解决某个特别难的问题，却会在另一个地方出错？数学能力也不是简单地在少数地方出现尖峰，它的差异可能细到不同问题层次。

如果看各种训练方法各自追求什么，就比较容易理解：基于人类反馈的强化学习（RLHF）希望模型的回答获得人的认可；RLVR 优化的是能通过程序验证的任务表现。这类任务之所以能用于 RLVR，本身就需要有可验证的结果。我们做的 RLCD，则是希望模型能可靠地供程序调用。

Swyx：我说一个实际使用中的观察，你看看对不对。拿到 Jev 的访问权限后，我第一天就用它试了很多任务。单步推理的效果非常好，可以说是顶尖水平；但需要连续推好几步时，表现就开始下降，而且步数越多，问题越明显。

Diogo Almeida：对。这又回到了实验问题：我们究竟能从模型中挖掘出哪些能力？我们希望尽可能多地释放它已有的能力，同时让能力分布更平滑、补上缺口，也增加新的能力。但归根结底，我们是在发掘这些高度浓缩的模型内核所具备的特性，再设法把不同特性组合起来。



目前有效的那部分能力，恰好可以用“System 1 式”来描述。这也是为什么我们没有选择让模型在字符串里写出很长的推理过程。我认为，模型很擅长在内部完成推理，尽管这种能力还不完整，也不是对所有问题都有效。

Swyx：等一下，你说的“潜在推理”(latent reasoning) 是在字符串里推理？我以为它指的是模型内部的推理。我想确认一下术语。

Diogo Almeida：我印象中，模型内部那种形式以前被叫作“连续推理”(continuous reasoning)，但我也不能完全确定。过去之所以有人把另一种情况叫作“潜在推理”，是因为推理轨迹没有公开：对只看到答案的人而言，它相当于一个隐藏变量。

Swyx：所以“隐藏”的究竟是哪一部分，大家的用法也在变化。

Diogo Almeida：至少 OpenAI 和 Anthropic 的推理轨迹，现在仍有不向用户完整展示的部分。

Swyx：那 Jev 将来也不会做推理模型？毕竟显式推理似乎违背了你们强调的 System 1 路线。

Diogo Almeida：我的承诺是，为了做好面向机器的智能，采用必要的方法。我能想象某些推理方式没有那么慢、低效和脆弱；如果是这样，我们完全可能采用。我不会承诺永远只用某种技术方法。我坚持的是最终要优化什么、为用户创造什么价值。产品已经发布了，但我们仍觉得还有很多事要做。

Swyx：视觉能力也是一个目前缺少的重要部分。不过，它适合放进 System 1 模型吗？

Diogo Almeida：在我看来，各种能力都有可能加入。我们内部也在讨论一个更大的问题：是优先先为用户提供我们判断他们真正需要的东西，还是更快提供他们明确说自己想要的功能？过去两年我们在未公开产品时，主要走的是前一条路，因为我们确信这个方向有价值。但产品发布后，两者之间需要找到平衡。

上下文长度就是一个例子。据我观察，我们的模型在长上下文中保持能力方面表现很好。但行业里也常见另一种做法：用户想要更长的上下文，就继续把数字做大。另一方面，如果我们过于坚持“我们知道用户需要什么”，又可能走向替开发者做过多决定的方式，那同样不利于开发者。

我们不希望把判断模型能力边界的全部负担都推给开发者，也希望他们能作为有判断能力的成年人，自行做出知情选择。接下来要解决的是：功能以多快的速度发布，才能兼顾产品的可信度和开发者的自主权。

Swyx：这个权衡很合理。

Diogo Almeida：老实说，我们还没有答案，得边做边找。接下来几天，这可能就是我最需要反复讨论的问题之一。我们手里还有不少东西。之前没想到这次发布会有这么大的反响，我们原本还想着，之后要陆续发布一些后续成果。

11 发布前没人看懂，发布后用户开始教他们怎么用

Swyx：我不太信你们完全没预料到发布后的反响。过去两个月，我从没见过你这么全力投入一件事。

Diogo Almeida：那也得感谢我的幕僚长，是他让我必须全力投入。我以前以为自己已经够努力了，这次才发现还能更拼。

Swyx：你甚至来参加我们的写作工作坊。当时我还在想：“你怎么也来了？”能看出来，你们为发布做了很认真、很有意识的准备。成果也体现出来了，恭喜。

Diogo Almeida：谢谢。我希望接下来还能保持这种投入。我们已经通过了技术圈的几道检验，但后面还有很多关要过，我很期待继续做下去。

Swyx：在聊 TypeSafe 以外的话题之前，还有什么关于这次发布的内容，你觉得被低估了，或者大家误解了？比如产品的使用模式、模型能力不均匀的问题？

Diogo Almeida：每个话题我都能讲很久，但还是克制一下吧。我想特别提一下我们的示例手册（cookbooks）。团队在里面花了很多心思，内容也很实用。我们原本考虑把不少例子放进发布博客，但那样文章会太长，也太偏向深度用户。

说实话，发布前我们很担心：这个工具听起来像是从外星来的，别人为什么需要它？我们觉得，教会大家理解这个新方向，可能会成为最大的障碍，所以在说明和示例上投入了很多。现在看来，用户已经做出了远超我们预期的东西，这个障碍或许没那么大了。

Swyx：用户反过来教你们怎么用自己的模型。

Diogo Almeida：没错。有些用户案例甚至比我们的演示更精彩。我看到一些计算机操作方面的用法时就想：如果发布时拿这个做演示该多好。至于示例手册，我们确实认真做了，不是随手生成的一堆内容。每个例子都有实际价值，很多来自我们帮客户解决真实问题的经历。

Swyx：发布前，你们做了多少用户验证？当时具体是什么情况？毕竟那时接触到的人还远没有现在这么多。

Diogo Almeida：坦白说，发布前的反馈不太好。团队里不做技术的同事尤其担心：很多人既没看懂，也不觉得自己需要它。大家会问，我们卖的会不会只是“维生素”，而不是能解决眼前痛点的“止痛药”？是不是还得配备前沿部署工程师（FDE），替客户把周边软件都写好，产品才能发挥作用？

发布前，我们几乎没有收入。技术团队相信它，因为我们看得到它在多个维度上的计算特性，也相信它有很大潜力。但我自己也很害怕，这正是我那段时间拼命投入的原因。

让超过一半试用者理解它在做什么都很难。少数看懂的人会说：“这很酷，但我们怎么通过采购流程？”整个过程并不轻松。后来我们决定，先面向开发者发布。开发者会找到用途，其他人看到实际效果后，也会想要用。

我不想嘲笑那些后来改变看法的人，情况确实变了。但这段经历让我重新思考“产品市场匹配”这个概念。发布前，我们拿着产品去问潜在用户：“你们要用吗？”他们说：“不知道它能不能解决我们的问题。”发布后，需求突然爆发，大家开始问：“能把我们的速率限制再提高吗？你们算力不够的话，我们甚至可以给你们提供 GPU。”

营销当然起了一定作用，但我觉得更关键的是，一群真正喜欢这个方向的开发者先用起来了，他们做出的东西又感染了更多人。

Swyx：很多公司也是从开发者起步，最后转向大企业客户。



Diogo Almeida：对。但我希望我们始终记得最早支持我们的开发者，而不只是把他们当作进入企业市场的跳板。我甚至在想，能不能推出一些对开发者比对企业更有利的东西，让他们获得更多能力。我已经有些想法了，虽然这么做可能很不寻常。

我也不知道还能怎样表达感谢。昨天我去染发，也是想借这个机会继续和他们交流。公司正处在一个重要阶段，如果这时候反而不再跟开发者说话，我会觉得不对劲。

Swyx：所以你的头发是这么回事。

Diogo Almeida：对。请大家以后拿这些话要求我。我希望自己能坚持原则；如果哪天我变了，尽管指出来。

12 从暗数据到智能软件：Jev 瞄准四类核心场景

Swyx：我想给大公司的开发者一些具体方向。即使他们现在没有做这类项目，也可以知道有哪些用法值得去看。

Diogo Almeida：我本来想先聊编程 Agent，不过还是先把主要的几类用法讲完。这些方向是我们发布前就从基本能力出发梳理出来的。

第一类是我们所说的“暗数据”(dark data)。很多企业积累了海量数据，却因为用大语言模型分析的成本太高，一直没能充分处理。大型企业很需要这类能力：它们有成堆想分析的数据。对数据科学家来说，这也是很有吸引力的场景。我认为，暗数据分析和编程 Agent 很可能是调用量最大、商业价值也最高的两个方向。

第二类是实时场景，需要在处理流程中迅速得到模型的判断。这类公司的 CEO，或者至少 CTO，大概都清楚：响应时间每减少 10 毫秒，产品体验能改善多少。

Swyx：电商尤其如此。

Diogo Almeida：对，各类 AI 助手也需要这种能力。据我从团队那里听到的反馈，这些用户很喜欢 Jev；不过我现在不直接负责客户沟通。我自己还特别期待它用在游戏里，比如玩家用指令指挥队伍作战的自动或半自动对战游戏。那会很有意思，不过在我还有工作的时候，先别把它做得太厉害（笑）。

第三类，我们叫“验证一切”：用它检查其他大语言模型的调用和输出，有点类似给 AI 系统增加可观测性。这里有个具体的使用技巧：如果你有一大段状态信息，想并行检查其中很多内容，可以给每条消息加上 ID，再针对各个 ID 分别提问。这样，较长的状态信息只需传入一次，就能围绕其中的消息提出许多问题，成本会低一些。

Swyx：按 System 1 和 System 2 的分工来想，这似乎意味着：每调用一次推理模型，就可以搭配一次、十次，甚至上百次 Jev 调用。

Diogo Almeida：也许，但我更希望用户花得更少。比如把原本需要的推理模型调用减半，再为每次调用配上若干次 Jev 判断。具体怎么组合，取决于它能否解决原来解决不了的问题。

第四类是“智能软件”：让智能判断成为软件自身的一部分，使程序能够组合出以前做不到的行为。有人尝试把 Jev 融入编程语言，做出了很有意思的东西。我们现在的基础设施还处在早期；如果能更方便地发放使用额度，我很想给这些项目提供支持。

这四类是我们事先梳理出的主要方向。计算机操作也是后来出现的用法，可以归到实时场景里。如果它能稳定工作，我会非常兴奋。这个方向多少有些出乎我们的预料，我也觉得模型在这方面还有很大的改进空间。

Diogo Almeida：编程 Agent 领域眼下正发生一件让我意外的事。在我印象里，Claude Code 和 Codex 大概是目前最领先的两个产品，不过我没有持续追踪排名。它们基本是围绕“单模型”来设计的，这很合理：过去开发者选择编程 Agent，主要是在功能相近、能力水平不同的模型之间做选择。

现在，开放式的编程 Agent 项目都在兴奋地尝试接入 Jev。我相信他们也在试各种别的组合。目前这些 Agent 的能力大致接近，毕竟只靠一个循环调用模型的框架，能做出的差异有限。一旦有人找到某个只有自己的 Agent 能做好的关键用例，用户就会涌过去；但其他开放式项目也能很快借鉴。Claude Code 和 Codex 则是围绕单模型构建的，我很好奇它们会怎么应对。

我个人很希望能与它们集成，也希望 Jev 能接入各种产品。它们以后也许会推出与我们竞争的能力，我不知道。但作为一家基础设施公司，我的工作不是预先替行业决定大家该用谁，而是让更多人能够使用我们的能力。这会让编程 Agent 市场变得很有意思。

我还写了一份内部文档，整理了一些可能适用于编程 Agent 的设计模式，正在请团队审阅，希望很快能分享出来。这个领域有太多值得探索的东西了。如果我不是正忙着做 Jev，我现在也很想亲自去做编程 Agent 实验。

Swyx：编程 Agent 公司应该也愿意和你们一起探索。我觉得 Claude Code 和 Codex 仍有使用 Jev 的空间。接下来暂时跳出 TypeSafe，聊聊你对 AI 研究方向，尤其是安全与对齐的看法。

13“放慢前沿进展”之外，还有别的研究路线吗？

Swyx：我最近参加了一场研究者聚会，大家确实担心技术发展的速度。有一种观点认为，公众还没有准备好，因此前沿实验室应该放慢脚步。你怎么看？

Diogo Almeida：这是个容易引起争议的话题，不过我愿意谈。我还想专门写一篇更完整的回应，这里先讲一个简短版本。

我觉得，“要不要放慢前沿进展”的讨论，往往默认了一件事：所有人都必须继续做 RLVR，而且要不断加大力度。但我不这么认为。就 Jev 这类模型想完成的任务而言，我认为最合适的 RLVR 用量甚至可能是零。

RLVR 也不能只从“奖励可验证”来理解。在推理模型兴起之前，人们就尝试过用可验证结果做强化学习，并没有因此自然得到今天的效果。训练任务的整体形态同样重要。我想起一段经历：RLHF 刚开始受到关注时，有几条后来被统称为“后训练”的研究路线同时在推进。指令遵循当时并不受所有人重视，因为评估它很麻烦。我曾跟预训练团队说，这里面有关键价值；他们的顾虑是，模型实验做得那么频繁，难道每次都要等人工评测结果才能决定选哪个模型？

当时，代码生成也获得了很多资源。团队尝试用单元测试的结果给模型做强化学习，取得过一些成果，但光有可验证的测试结果，并不能自动带来今天这种推理能力。因此，RLVR 的效果不只是奖励函数带来的，也和模型被允许经历怎样的推理、采取怎样的中间步骤有关。为了让模型解出最难的问题，研究者会给它很大的行动空间。

这也是我对“放慢前沿进展”论述的疑问。有些人会说，也许问题出在沙箱隔离不够。那当然可能是问题，也可以改进。但实验室之所以给模型更大的行动空间，是因为这样可能让它在任务上



更强；如果增加约束，又可能损失一部分表现。我认为，这里有一些本该由研究者自己承担的设计选择，却被表述成了发展 AI 不可避免的前提。

Swyx：如果先接受“必须沿着这条路线继续加大投入”的前提，再推导出风险会增长，逻辑上是说得通的。但前提本身还有其他选择。

Diogo Almeida：正是如此。真正重新定义任务或研究目标的路线很少见，所以研究者容易沿着已有方向继续推进。我觉得现在对其他可能性的思考还不够开放。

公众不该为这种局面负责。他们不是研究这件事的专家，通常只能相信 OpenAI、Anthropic 等实验室已经在尽最大努力；更了解技术上还有哪些选择的，是研究者自己。

Swyx：你们做的事，也是在让大家看到另一条路。

Diogo Almeida：我会尽力。但我的目标不是说服其他实验室改变路线，而是让软件工程师重新看到希望，开始自动化那些他们一直想自动化的工作。

我曾写过一篇关于理想中 AI 未来的文章，团队最后没让我发表。里面有很多很小、但会改变体验的设想。刚才那个用语音操作电脑的演示就是一例：过去计算机太机械、太按字面执行指令了；如果它能更好地理解人的意图，很多操作都会顺畅得多。

我不想承诺这些很快就會实现。但我们会尽力推动，让智能逐渐进入软件的各个角落。

Swyx：回到训练方法。你深入参与过后训练，怎么看“中训练”(mid-training)？我们好像还没聊过这个。

Diogo Almeida：训练的不同阶段更像是一条连续的谱系，没有那么清楚的分界。

Swyx：可以理解成一种更复杂的课程学习？

Diogo Almeida：可以这么说。中训练也有成本上的考虑：你不必为了增加某些能力，从头再做一遍预训练。

我觉得智能在训练的每个阶段都有一些难以用简单规则概括的地方。观察数据、弄清能力是怎么进入模型的，是件很有意思的事；我们的数据团队很擅长这个，我自己没有他们那么擅长。我更习惯从整体结构上思考，也很喜欢听他们讲从数据里发现了什么。

比如，短期内可以通过微调让模型迅速表现出某种倾向；如果相关训练持续进行，某些能力就会逐渐更深入地进入模型，变得更稳固。我想找到的正是那些能稳定发挥的能力。前面说的 System 1 式能力，也属于这一类。所以我对中训练很感兴趣，也支持探索各种训练方式，看看还能从模型中释放出什么能力。但这些方法成本很高，我不会每一种都亲自做。

有件事我以前私下说过。我觉得，如果能对投资人说，也应该能公开对大家说：即使给我 10 亿美元，我也不会自己做预训练。我现在仍这么想。预训练太贵了。作为 AI 工程师，你可以拆解现有模型、组合不同能力，尝试各种办法。把模型像“弗兰肯斯坦”一样拼起来，也许不够优雅，但它能解决问题。总之，我会尝试预训练之外的办法。

Swyx：顺着这个思路，有一个值得讨论的问题：未来是把所有能力放进一个“超级模型”，还是继续把不同能力拆到不同模型里？OpenAI 曾经朝全能模型（omni model）的方向走，GPT-4o 就是例子。但有一段时间，也能看到聊天优化模型和编程优化模型各走一条线。



Diogo Almeida：这其实是两个差别很大的问题，得拆开说。多模态是一回事：加入其他模态，有时会帮助模型，有时也会拖累它。比如，我看到有些团队似乎在减少对语音（speech）的投入。这里说的语音和更广义的音频（audio）不是一回事；语音能力目前似乎不太容易迁移到其他任务上。这个问题以后可能会解决，但得看实验结果。

我支持探索各种形式的智能，不过不能以为只要按缩放规律增加投入，问题就一定会解决。缩放规律最终要回答的是：投入增加后，能力实际上提高了多少？有些任务即使继续扩大规模，也可能达不到可用水平。据我了解，计算机操作目前就还没有解决。我希望我们能在其中发挥作用，但也可能无论收集多少数据都不够，需要改进方法。

所以，问我是否支持全能模型，我的回答是：我支持探索各种能力。但你刚才提到的另一个问题更值得展开——后训练怎样影响模型原本具备的能力。我很不喜欢把智能训练得支离破碎。

以聊天优化为优先的训练，往往高度依赖 RLHF，也容易带来人们经常抱怨的问题：迎合用户、过度自信、幻觉。甚至在 LM Arena 这类评测中表现讨喜的回答风格，也可能是训练目标塑造出来的：大量加粗、斜体、表情符号；不直接回答问题，而是写一大段，再反问用户一句，让对话显得更像真人。

我认为，这和让模型更准确地发挥已有能力是两回事。让模型生成一段自由文本，本来就有很多不确定性；训练过程中，模型可能学会用非常确定的语气维持某种回答模式，避免明显跑偏后受到奖励模型的惩罚。这会改变它的概率分布，也会与推理能力相互影响。模型可能学会一些取巧的方式，把目标完成了，判断能力却未必因此变得更稳固。理解这些细微变化，是研究智能的重要部分。至少在我还在 OpenAI 时，我觉得这方面得到的研究还不够多，大家更多是在优化“聊天”这个目标。

Swyx：一旦确定目标，团队就会集中优化它。

Diogo Almeida：对。有人会说，那就同时优化两个目标。但在我看来，把模型分别往不同目标上拉，本身就可能造成能力分裂。

Swyx：可是，你把能力分成 System 1 和 System 2，某种程度上不也是在拆分智能吗？只不过你不认同别人拆分的方式。

Diogo Almeida：我觉得两者不一样。我们并没有把 System 2 任务扔掉。开发者可以拿 Jev 去尝试这类任务，而对于模型没有把握的问题，一个合理的结果就是表达“不确定”。它应该给出较低置信度，清楚体现不确定性；某些启发式方法也许还能改善表现。我们同样关心这些任务，只是我认为，长链条的 System 2 式推理并非这个模型最自然、最稳定的能力形态。

我也不想为了塑造产品身份而改变模型的回答。比如有人问它是谁，我不会特意训练它回答“我是 TypeSafe 的 Jev”。那会把一个品牌要求写进模型的判断里。我更希望它依据学到的信息给出正确回答，让能力保持平滑、可预测。

Swyx：模型身份对直接面向用户的产品来说，可能是个特殊问题。

Diogo Almeida：对，第一方产品需要考虑身份。但如果提供的是 API，情况就不同了。开发者用模型做自己的聊天产品，通常希望它呈现的是自己的产品身份，而不是回答“我是 ChatGPT”。

Swyx：为了让开发者更容易把 Jev 用进产品，你们也提供了 Jev Skill，让编程 Agent 知道该怎么与 Jev 配合。



Diogo Almeida：对。

14 从 InstructGPT 到 TypeSafe AI 创业

Swyx：最后问几个问题。回头看，你做这件事大概有两年多了？

Diogo Almeida：如果从公司成立算起，是两年多；但对我来说，更像是一段持续了四年的经历。

Swyx：我想起有一年感恩节，你取消了其他安排，说大家都在休假，正好可以用空出来的 Edge GPU，集中做一次实验。那还是 TypeSafe 成立之前吧？

Diogo Almeida：对，那段时间挺有意思。是不是正好赶上 OpenAI 那次管理层风波？我有点记不清了。

Swyx：对，时间差不多。

Diogo Almeida：那就对上了。那场风波挺让人心烦的，不过我们今天没时间展开聊。

Swyx：你说让人心烦的是那场风波，还是那次实验？

Diogo Almeida：是那场风波。也许下次聊天时，我再讲当时的事。不过，Jev 想解决的问题，在 ChatGPT 发布前就已经萦绕在我脑子里了。当时我看到 ChatGPT 团队的工作，觉得他们选对了任务。他们做了一件很多 AI 研究者不擅长、但优秀产品团队很在意的事：认真打磨用户体验。在当时的 OpenAI，这种人并不多；ChatGPT 团队在这方面做得很好。

Swyx：你说的这段经历，可以追溯到 GPT-3 向 GPT-3.5 演进的时期。当时还有 AI Dungeon 这样的应用，也是一个你们事先没想到的用例。

Diogo Almeida：对。我当时也很努力地推动 InstructGPT 上线。早期有些版本甚至用了我自己设计、但后来没有公开的训练算法，因为清理 PPO 数据太慢了。我当时想：效果已经这么好，应该尽快让用户用上。

它上线后，很快就在当时的大语言模型使用市场里占了相当大的份额，按我们的估计大约有 50%。我还花了很多精力，确保发布视频里的展示都是真实的。当时我甚至认真想过：一个模型能这样理解并执行指令，这算不算 AGI？现在看，显然不算。但我觉得每个人都值得想一想，为什么它看上去这么聪明，却仍然不是 AGI。

后来，我看到它主要被用于营销文案写作，例如 Jasper AI、Copy.ai，以及大量填充网页的文字。我们一度担心，自己是不是让互联网变得更糟了。于是我重新思考：模型明明表现得很聪明，为什么没有创造出我们期待的价值？还缺了什么？

我试着从一场由 AI 推动的经济变革倒推：如果 AI 以 API 的形式存在，未来主要是谁在调用它——人，还是代码？我的判断是，绝大多数调用会来自代码。但当时的优化几乎都面向人与模型的对话。那一刻我意识到，应该把“让程序能够可靠地调用智能”作为目标。

我写了一份文档，也跟 Sam 聊过。他觉得这个方向很好，建议我去做。我当时的反应是：“可我还本职工作啊。”

Swyx：Sam 都让你去做了，那就去做啊。



Diogo Almeida：当时我还觉得，这个方向太显而易见了，Anthropic 肯定已经在做，我们可能已经晚了。我那时的看法是，OpenAI 往往更擅长追赶已有方向。比如我认为 ChatGPT 与 Anthropic 此前内部做过的聊天产品有相似之处，只是后者当时没有发布。

Swyx：对，之前还有在 Slack 里使用的 Claude。不过，推理模型这条线上，OpenAI 算是较早推出产品的。

Diogo Almeida：研究成果很出色。至于它是否对应用户当时最迫切的产品需求，我不太确定。编程 Agent 这边，Claude 也做出了重要的产品。

和 Sam 聊完后，我先回去继续做原来的工作。后来，指令遵循团队觉得这方面已经取得了很大进展，我也开始想下一步做什么，于是试着研究那个面向程序调用的方向。我原以为训练模型、验证想法只要一周，最后却花了好几年。

中途有一次，我看到了初步有效的迹象。它当时显然还不能直接部署，否则我们早就发布了。但我很想知道，如果在研究上全力投入，这条路线最终能走到哪里。我也想过：如果 AI 最后进入新一轮低谷，而我明明看到了这个问题却没有认真尝试解决，我会觉得自己有责任。

我和其他公司聊过，提出想建立一个研究这个方向的实验室，也得到了一些兴趣。但当我问他们，加入现有公司和自己创业，哪一种能推进得更快，他们的回答是“创业”。于是我决定自己做。

Swyx：然后你联系了 Eric 和 Sasha？

Diogo Almeida：我先联系了 Eric（译者注：Erik Gafni，现任 TypeSafe AI 首席技术官）。找 Sasha（译者注：Sasha Sheng，现任 TypeSafe AI 首席运营官）时，我起初并不是想拉她入伙。我只是问：“是不是我想错了？是不是因为待在 OpenAI 太久，我没看到外面其实已经有解决方案？”

结果她说：“我加入。”我提醒她，当时她还在经营一家创业公司，她说正在结束那家公司。我劝她先认真想想，她想过之后，还是加入了。

两周之内，我们拿到了资金，也有人搬进我的公寓一起工作。我有点洁癖，那段日子对我来说可不轻松。此后我们继续做研究，终于得到了一些结果，证明这个方向有初步的可行迹象。

15 别再复制前沿实验室：先找到你的 North Star

Swyx：前面讲了这么多背景，我真正想问的是：如果现在有一位前沿实验室的研究者，和当年的你一样，觉得自己的方向得不到资金、资源或重视，你会建议他也出来创业吗？

Diogo Almeida：这问题很有意思。我得想想怎么说才不至于得罪太多人。

坦白说，除非这些新兴实验室背后有我不了解的经济逻辑，否则我不太看好其中的大多数。我看重的不是研究者履历本身，而是他们是否认真选择了值得解决的问题。我们当然需要研究者，但他们得真正关心自己研究的任务。对我来说，首先要找到一个值得长期投入的 North Star（核心目标），然后围绕它做真正有用的事。光有漂亮的研究背景，通常不会自动创造价值。

我相信应该有清楚的目标，去做有用的事。如果一家新实验室有明确方向，我非常支持；但如果只是从头重复已有工作，拿到资金后做各种实验，真正推进前沿的机会又很小，我看不到它



创造多少价值。据我与一些新兴实验室交流的经验，有些团队还没有想清楚自己要往哪里走。

所以建议取决于你为什么离开。如果只是想自由地尝试研究课题，现有实验室可能仍是最合适的地方。我更希望大家带着要解决的问题出发。问题可以是探索性的，但最好有自己认同的原则和方向。如果你确信自己找到了一个值得投入的新任务，那我会强烈支持你去做。我们需要有人打破研究圈里越来越一致的思路。

Swyx：像一种“蜂群思维”。

Diogo Almeida：对。“放慢前沿进展”的讨论，也部分来自对 AI 的一种特定想象：模型聪明得惊人，能力却非常不均匀，因而带来特殊风险。但这只是发展 AI 的一种路线，不能把它当成唯一可能。探索不同的技术方向，本身就有价值。

Swyx：公平地说，我想准确转述此前与我交流过的 OpenAI、Anthropic，还有 SpaceX 人士的意思：他们谈放慢进展，考虑的不只有灾难性风险，也有政治层面的因素。

Diogo Almeida：政治层面的事，就超出我的专业范围了。

Swyx：我当时听完，也意识到他们讨论的时点可能和 2028 年选举有关。

Diogo Almeida：我真希望没听到这一层。感觉很糟。

Swyx：先说明一下，这只是当时那场讨论中部分人的说法，不能代表整家公司。

Diogo Almeida：明白。也许我是个天真的技术人员，但听到这些，我多少有点失望。

Swyx：随着技术发展，谁来执政、怎样制定监管规则，确实会越来越重要。实验室也需要认真考虑这些问题。

Diogo Almeida：这点我完全同意。我担心的是，为了达到某个自认为正确的目标，在公开沟通时把话说得过于确定，或者让公众产生误解。我不想在这里展开谈政治。我的原则是，即使出发点被认为是为了公共利益，也应该尽量准确地说明事实和不确定性。

Swyx：就我听到的讨论而言，我不觉得他们是在误导公众。他们更多是在解释，为什么现在提出放慢进展。

Diogo Almeida：我理解。但如果公开强调的风险理由，与实际推动这件事的目标并不完全一致，我仍觉得其中有值得说清楚的地方。希望我们以后尽可能不卷入这样的事。公司变大后，也许很难完全置身事外，但我还是想守住自己作为技术人员的出发点。

Swyx：那我开个玩笑：让 Jev 竞选总统吧，我说不定比信任自己的决定更信任它。好了，玩笑先放一边。你们已经选定了自己的 North Star：让 AI 可靠、可编程、可组合，还要便宜。如果把机会留给其他人，你最希望他们去解决什么问题？

Diogo Almeida：这问题太好了。我先说一个好玩的，再说一个我觉得很有研究价值的。

第一个是游戏。如果游戏里的角色和世界能更智能，会非常有趣。我看过 Ali 做的《Doom》演示，玩家可以给 NPC 执行一些操作。那还只是概念验证，却已经让我想到很多可能。我很喜



欢《星露谷物语》；它的世界相对静态，仍然很吸引人。如果角色能对游戏状态作出更多反应，就可以产生很多新的故事。

也不必在游戏的每一帧都调用 Jev，那样可能太贵了。哪怕只是在 NPC 的状态机里加入一些智能判断，也有机会把游戏世界做得更生动。我有点遗憾自己现在没法亲自做这些项目。

Swyx：可以让别人来做，你再给他们反馈。

Diogo Almeida：对。另一个我特别想看到有人探索的方向，是摆脱 KV Cache 对编程 Agent 设计的限制。我写过一篇文章，叫《KV Cache Rules Everything Around Me》，就是想解释现有编程 Agent 和 KV Cache 是怎么相互影响的。它能解释很多现象：为什么模型路由很难，为什么子 Agent 的效果常常不如预期，为什么上下文压缩也是个棘手问题。

我还写了一份关于新设计模式的文档，希望团队审过后能发布出来。我想把自己的想法抛给大家，请大家试试看：如果不再被现有的 KV Cache 使用方式绑住，编程 Agent 还能怎么设计？

Swyx：你说的“绑住”，是指它限制了模型选择？

Diogo Almeida：这是一方面。为了高效利用 KV Cache，系统往往要持续在已有上下文后面追加内容，也就容易固定在同一个模型上。这样一来，设计会逐渐偏离软件工程里熟悉的做法，比如明确管理状态、抽象问题、拆解任务。

举个例子，为什么不能把简单的任务交给一个更便宜的子 Agent？难点之一是状态交接：如果把父任务积累的大量上下文都传过去，子 Agent 光是读取这些信息就可能花掉不少成本。那能不能更聪明地决定，它到底需要哪一部分状态？

我觉得这里有很多值得研究的编程模式。可以给状态明确标注含义，把任务组织成有层级的子任务。编程 Agent 本来就在逐个处理子任务；一个子任务结束后，为什么一定要把它接触过的全部状态都塞回父任务？需要信息时，为什么不能在子任务树里检索相关上下文？

如果访问上下文的成本足够低，还可以更方便地回看历史记录。现在 Agent 经常像每次都从零开始工作，然后我们又要单独解决“持续学习”的问题。其中一部分其实是记忆管理：信息并非完全不存在，而是系统不知道怎样有效找到它。

并行工作的子 Agent 也一样。如果它们的状态保存在计算机内存里，为什么不能有选择地读取彼此的进展？哪些内容正在写入、哪些可以读取，都可以更清楚地管理。多个 Agent 协作时，也许还能用比简单加锁更灵活的方式协调：“你现在在改什么？我在改什么？谁应该先写？”

Swyx：让 Jev 来决定谁先拿到锁？

Diogo Almeida：多 Agent 协作里确实可能有这样的用法。还有一些 Agent 只负责读取状态，比如向用户汇报其他 Agent 的工作进展。它不需要看完所有探索过程，只要找到已经写入的结果、相关子任务和当前状态就够了。

如果有人愿意投入足够多的时间，重新思考编程 Agent 怎样保存、查找和共享状态，我觉得能做出很有意思的东西。这是我特别希望看到的方向。

Swyx：你可以看看 PrimeAgent。它和递归语言模型（RLM）相关的工作有结合。我们刚和参与这方面研究的 Alex 聊过。这条路线已经有人在做，只是目前还不算热门。

Diogo Almeida：那很好。我希望更多人去试各种不寻常的想法。我不能保证它们一定有效，但从技术上看很值得探索。等我们建立起发放使用额度的机制，也希望能支持做这些实验的人。

Swyx：以后你们或许还能直接资助相关研究。也恭喜你整个团队走到今天，从我第一次认识你到现在，你和团队都走了很远。

Diogo Almeida：我希望自己还是原来那个人。

Swyx：我觉得是。不过现在的你比我以前见过的任何时候都更有劲头，因为你找到了自己的使命。过去很多年，你一直在指出问题，但还没有拿得出手的解决方案；后来你有了大致方向，又花了很长时间把它做出来。

Diogo Almeida：这也是因为我觉得自己完全没有创业者的性格。我不喜欢创业，从没想过当 CEO。做一次已经够难了，很难想象有人愿意做第二次。最初融资时，有位投资人问我仰慕哪位 CEO，我的第一反应是：“我为什么要仰慕他们？”

无意冒犯谁。我遇到过很多很好的人，只是一些最出名的人似乎也有不少不愿示人的问题。在 OpenAI 时，我确实觉得自己很难推动想做的事。现在至少有了一点证据，说明我们的方向可能走得通，我也更容易把当时的想法讲清楚。

那时候，周围很多讨论都是“怎么把 ChatGPT 放进更多产品”“怎么让 ChatGPT 更适合开发者”。我却一直在想：现有的函数调用接口为什么要这样设计？开发者怎么用它来可靠地控制程序？

Swyx：像是在已有的变通办法上，再叠加一层变通办法。

Diogo Almeida：问题还不止于此。我当时常说：如果函数调用不能为每个函数提供相应的 logit bias，让开发者知道或调节模型选择它的倾向，就不要让我参与这个项目。我的要求并不复杂。

Swyx：你想要的是某种与置信度接近、但未必经过校准的信号。

Diogo Almeida：对，或者至少给出选择某个函数的概率。开发者需要控制模型在不同情况下采取什么行动，包括允许还是拒绝。同样是拒绝，迪士尼和 AI Dungeon 想设的门槛肯定不一样。可如果接口只让开发者通过提示词反复请求模型“请在这种情况下拒绝”，那怎么能算一个好的程序接口？大家已经这样用了很多年。

现在的 Skill 也有类似问题。现有编程 Agent 对各自的运行框架适应得很好，但能力并不均匀，使用外部工具和 MCP 时往往没那么可靠。如果某个工具对业务很重要，开发者为什么不能明确提高模型调用它的倾向？现在往往还是得在系统消息里反复要求。这让我觉得很不合理。

Swyx：我明白你的意思了。很高兴能在节目里和你聊这些，也期待你们接下来发布的東西。

Diogo Almeida：谢谢。接下来可能比你想象的还快。

Swyx：你们现在还在招数据、基础设施、市场和社区方面的人？

Diogo Almeida：都在招。我自认为还能胜任创始营销负责人，但团队会让我别再抢这份活，好好做 CEO。事情一下子多起来之后，你很快就会学会授权。

我们也在招负责模型能力的人。他们的工作很大一部分与数据有关。我希望团队充分认可这类工作的价值，让直接改进模型能力的人得到应有的重视，同时又不在于团队里制造奇怪的等级。



说到文化，我挺高兴同事们不会因为我是 CEO 就事事顺着我。他们会开我的玩笑，也会直接反驳我。我觉得这是个好信号。

另外，我们还需要平台工程师，把 Jev 部署到更多地方。对我们来说，服务器的位置尤其重要，因为网络传输时间会直接影响延迟。现在欧洲还没有我们的服务器，欧洲用户体验到的速度提升可能只有约三倍，而不是本来有机会达到的约一百倍，这让我很遗憾。

Swyx：没关系，欧洲的生活节奏也慢一点。

Diogo Almeida：这话可是你说的，不是我说的（笑）。总之，我们希望把服务部署到更多地区。如果“每秒能获得多少智能”对用户很重要，我们就会把服务部署到更多地区。开发者在我们的能力之上构建产品，我非常在意他们的使用体验。

我们也在招人，继续开发 Jev 之外的能力。公司的目标并不是只做 Jev 这一款模型，更不想只靠一种简单模型走到底。未来可能需要一个像 AWS 一样的平台，提供形态各异的智能能力。我希望我们朝这个方向走。

Swyx：而那个平台会是你们，对吧？

Diogo Almeida：现在就说一定是我们，太自大了。但我会尽一切努力去做。我觉得这会很有意思：我们现在探索的只是 System 1 式智能中的一层。如果把它类比为 TCP，上面还能建立许多层能力。

Swyx：对，还有很多层。我甚至提过 Temporal，也许它能算传统七层模型之外的“第八层”。不过这个话题我们又能聊很久。你该回去工作，或者好好睡一觉了。谢谢你来。

Diogo Almeida：谢谢邀请，聊得很开心。我很期待接下来要做的事。

本文来源：[极客邦科技InfoQ](#)

风险提示及免责条款

市场有风险，投资需谨慎。本文不构成个人投资建议，也未考虑到个别用户特殊的投资目标、财务状况或需要。用户应考虑本文中的任何意见、观点或结论是否符合其特定状况。据此投资，责任自负。

 38  收藏

分享到:  

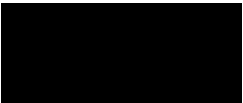
写评论

请发表您的评论

 表情  图片

发布评论

1 条评论



正在加载 .

